threading — Paralelismo basado en hilos

Código fuente: Lib/threading.py

Este módulo construye interfaces de hilado de alto nivel sobre el módulo de más bajo nivel _thread. Ver también el módulo queue.

Distinto en la versión 3.7: Este módulo solía ser opcional, ahora está siempre disponible.

Nota

Aunque no están listados en lo que sigue, los nombres en camelCase usados para algunos de los métodos y funciones de la versión Python 2.x todavía son soportados por este módulo.

CPython implementation detail: In CPython, due to the Global Interpreter Lock, only one thread can execute Python code at once (even though certain performance-oriented libraries might overcome this limitation). If you want your application to make better use of the computational resources of multi-core machines, you are advised to use multiprocessing or concurrent.futures.ProcessPoolExecutor. However, threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously.

Este módulo define las siguientes funciones:

threading.**active_count**()

Retorna el número de objetos Thread actualmente con vida. La cuenta retornada es igual al largo de la lista retornada por enumerate().

threading.**current_thread**()

Retorna el objeto Thread actual, correspondiente al hilo de control del invocador. Si el hilo de control del invocador no fue creado a través del módulo threading, se retorna un objeto hilo *dummy* con funcionalidad limitada.

threading.**excepthook**(*args*, /)

Gestiona una excepción lanzada por Thread.run().

El argumento *args* posee los siguientes atributos:

- *exc_type*: Tipo de la excepción.
- *exc_value*: Valor de la excepción, puede ser None.
- *exc_traceback*: Rastreo de la excepción, puede ser None.
- *thread*: El hilo que ha lanzado la excepción, puede ser None.

Si *exc_type* es SystemExit, la excepción es silenciosamente ignorada. De otro modo, la excepción se imprime en sys.stderr.

Si esta función lanza una excepción, se llama a sys.excepthook() para manejarla.

threading.excepthook() se puede sobrescribir para controlar cómo se gestionan las excepciones levantadas por Thread.run().

Guarda *exc_value* usando un *hook* personalizado puede crear un ciclo de referencias. Debe ser aclarado explícitamente que se rompa el ciclo de referencias cuando la excepción ya no se necesite.

Guarda *thread* usando un *hook* personalizado puede resucitarlo si se asigna a un objeto que esté siendo finalizado. Evítese que *thread* sea almacenado después de que el *hook* personalizado se complete para evitar resucitar objetos.

Ver también

sys.excepthook() gestiona excepciones no capturadas.

Nuevo en la versión 3.8.

threading.**get_ident**()

Retorna el "identificador de hilo" del hilo actual. Éste es un entero distinto de cero. Su valor no tiene un significado directo; ha sido pensado como una *cookie* mágica para usarse, por ejemplo, en indexar un diccionario con datos específicos del hilo. Los identificadores de hilo pueden ser reciclados cuando se abandona un hilo y se crea otro hilo.

Nuevo en la versión 3.3.

threading.**get_native_id**()

Retorna la ID de Hilo (*Thread ID*) nativo integral del hilo actual asignado por el *kernel*. Ella es un entero distinto de cero. Su valor puede utilizarse para identificar de forma única a este hilo en particular a través de todo el sistema (hasta que el hilo termine, luego de lo cual el valor puede ser reciclado por el SO).

Disponibilidad: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX.

Nuevo en la versión 3.8.

threading.**enumerate**()

Retorna una lista de todos los objetos tipo Thread actualmente con vida. La lista incluye hilos demonio, objetos hilo *dummy* creados por current_thread(), y el hilo principal. Excluye hilos terminados e hilos que todavía no hayan sido iniciados.

threading.**main_thread**()

Retorna el objeto Thread principal. En condiciones normales, el hilo principal es el hilo desde el que fue inicializado el intérprete de Python.

Nuevo en la versión 3.4.

threading.**settrace**(*func*)

Establece una función de traza para todos los hilos iniciados desde el módulo threading .
La *func*se pasará a sys.settrace() por cada hilo, antes de que su método run() sea llamado.

threading.**setprofile**(*func*)

Establece una función de perfil para todos los hilos iniciados desde el módulo threading.
La *func*se pasará a sys.setprofile() por cada hilo, antes de que se llame a su método run().

threading.**stack_size**([*size*])

Retorna el tamaño de pila usado para crear nuevos hilos. El argumento opcional *size* (tamaño) especifica el tamaño de pila a ser utilizado para hilos creados posteriormente, y debe ser 0 (usar el valor por defecto de la plataforma o el configurado) o un valor entero positivo de al menos 32.768 (32KiB). Si no se especifica *size*, se usará 0. Si no existe soporte para cambiar el tamaño de pila, se lanzará un RuntimeError. Si el tamaño de pila especificado es inválido, se lanzará un ValueError y el tamaño de pila no será modificado. El tamaño mínimo de pila actualmente soportado es de 32KiB para garantizar suficiente espacio de pila para el intérprete mismo. Nótese que algunas plataformas pueden tener restricciones particulares de valores para tamaños de pila, como requerir un tamaño de pila > 32KiB, o requerir una asignación en múltiplos del tamaño de página de la memoria del sistema. Debe consultarse la documentación de cada plataforma para mayor información (páginas de 4KiB son comunes; se recomienda el uso de múltiplos de 4096 para el tamaño de pila en ausencia de información más específica)

Disponibilidad: Windows, sistemas con hilos POSIX.

Este módulo también define la siguiente constante:

threading.**TIMEOUT_MAX**

El máximo valor permitido para el parámetro *timeout* de las funciones bloqueantes (Lock.acquire(), RLock.acquire(), Condition.wait(), etc.). La especificación de un tiempo de espera mayor a este valor lanzará un OverflowError.

Nuevo en la versión 3.2.

Este módulo define un número de clases, las cuales son detalladas en las siguientes secciones.

El diseño de este módulo está libremente basado en el modelo de *threading* de Java. Sin embargo, donde Java hace de *locks* y variables condicionales el comportamiento básico de cada objeto, éstos son objetos separados en Python. La clase de Python Thread soporta un subdominio del comportamiento de la clase *Thread* de Java; actualmente, no hay prioridades, ni grupos de hilos, y los hilos no pueden ser destruidos, detenidos, suspendidos, retomados o interrumpidos. Los métodos estáticos de la clase *Thread* de Java, cuando son implementados, son mapeados a funciones a nivel de módulo.

Todos los métodos descritos abajo son ejecutados de manera atómica.

Datos locales del hilo

Los datos locales de hilo son datos cuyos valores son específicos a cada hilo. Para manejar los datos locales de hilos, simplemente crear una instancia de local (o una subclase) y almacenar los atributos en ella:

```
mydata = threading.local()
mydata.x = 1
```

Los valores de instancia serán diferentes para hilos distintos.

class threading.**local**

Una clase que representa datos locales de hilo.

Para más detalles y ejemplos extensivos, véase la documentación del módulo _threading_local.

Objetos tipo hilo

La clase Thread representa una actividad que corre en un hilo de control separado. Hay dos manera de especificar la actividad: pasando un objeto invocable al constructor, o sobrescribiendo el método run() en una subclase. Ningún otro método (a excepción del constructor) deberá ser sobrescrito en una subclase. En otras palabras, *solo* sobrescribir los métodos __init__() y run() de esta clase.

Una vez que un objeto *thread* es creado, su actividad debe ser iniciada llamando al método start()del hilo. Ésto invoca el método run() en un hilo de control separado.

Una vez que la actividad del hilo ha sido iniciada, el hilo se considerará "vivo". Deja de estar vivo cuando su método run() termina – ya sea normalmente, o por lanzar una excepción no manejada. El método is_alive() verifica si acaso el hilo está vivo.

Otros hilos pueden llamar al método join() de un hilo. Esto bloquea el hilo llamador hasta que el hilo cuyo método join() ha sido llamado termine.

Un hilo tiene un nombre. El nombre puede ser pasado al constructor y leído o cambiado a través del atributo name.

Si el método run() lanza una excepción, se llama a threading.excepthook() para gestionarla. Por defecto, threading.excepthook() ignora silenciosamente a SystemExit.

Un hilo puede ser marcado como un «hilo demonio». El significado de esta marca es que la totalidad del programa de Python finalizará cuando solo queden hilos demonio. El valor inicial es heredado del hilo creador. La marca puede ser establecida a través de la propiedad daemon o del argumento *daemon* en el constructor.

Nota

Los hilos demonio son detenidos abruptamente al momento del cierre. Sus recursos (tales como archivos abiertos, transacciones con bases de datos, etc.) pueden no ser liberados adecuadamente. Si se requiere que los hilos se detengan con gracia, háganse no-demoníacos y úsese un mecanismo de señalización adecuado tal como un Event.

Existe un objeto «hilo principal»; éste corresponde al hilo de control inicial del programa de Python. No es un hilo demonio.

Existe la posibilidad de crear «objetos de hilos *dummy*». Estos son objetos hilo correspondientes a «hilos extranjeros», que son hilos de control iniciados afuera del modulo *threading*, por ejemplo directamente de código en C. Los objetos de hilos *dummy* tienen funcionalidad limitada; siempre se consideran vivos y demoníacos, y no pueden se les puede aplicar el método join(). Nunca son eliminados, ya que es imposible detectar la terminación de hilos extranjeros.

class threading.**Thread**(*group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None*)

Este constructor siempre debe ser llamado con argumentos de palabra clave. Los argumentos son:

group debe ser *None*; reservado para una futura extensión cuando se implemente una clase ThreadGroup.

target es el objeto invocable a ser invocado por el método run(). Por defecto es None, lo que significa que nada es llamado.

name es el nombre del hilo. Por defecto, se construye un nombre único con la forma «*Thread-N*» donde *N* es un número decimal pequeño.

args es la tupla de argumento para la invocación objetivo. Por defecto es ().

kwargs es un diccionario de argumentos de palabra clave para la invocación objetivo. Por defecto es {}.

Si no es None, *daemon* establece explícitamente si el hilo es demoníaco. Si es None (el valor por defecto), la propiedad demoníaca es heredada del hilo actual.

Si la subclase sobrescribe el constructor, debe asegurarse de invocar al constructor de la clase base (**Thread.__init__()**) antes de hacer cualquier otra cosa al hilo.

Distinto en la versión 3.3: Se agregó el argumento *daemon*.

start()

Inicia la actividad del hilo.

Debe ser llamada máximo una vez por objeto hilo. Se encarga de que el método run() del objeto sea invocado en un hilo de control separado.

Este método lanzará un RuntimeError si se llama más de una vez en el mismo objeto hilo.

run()

Método que representa la actividad del hilo.

Se puede sobrescribir este método en una subclase. El método estándar run() invoca el objeto invocable pasado al constructor del objeto como argumento *target*, si lo hay, con argumentos posicionales y de palabra clave tomados de los argumentos *args* y *kwargs*, respectivamente.

join(*timeout=None*)

Espera a que el hilo termine. Esto bloquea el hilo llamador hasta que el hilo cuyo método join() es llamado finalice – ya sea normalmente o a través de una excepción no gestionada – o hasta que el tiempo de espera opcional caduque.

Cuando se presenta un argumento *timeout* y no es None, debe ser un número de punto flotante que especifique un tiempo de espera en segundos (o en fracciones de segundo) para la operación . Ya que join() siempre retorna None, se debe llamar a is_alive() después de join() para decidir si acaso caducó el tiempo de espera – si el hilo todavía está vivo, la llamada a join() caducó.

Cuando el argumento *timeout* no se presenta o es None, la operación bloqueará hasta que el hilo termine.

A un hilo se le puede aplicar join() muchas veces.

join() lanza un RuntimeError si se intenta unir el hilo actual ya que ello generaría un punto muerto. También es un error aplicar join() a un hilo antes de que haya sido iniciado y los intentos de hacerlo lanzaran la misma excepción.

name

Un *string* utilizado con propósitos de identificación. No posee semántica. Se puede dar el mismo nombre a múltiples hilos. El nombre inicial es establecido por el constructor.

getName()

setName()

Antigua API *getter/setter* para name; úsese en cambio directamente como una propiedad.

ident

El "identificador de hilo" de este hilo o None si el hilo no ha sido iniciado. Es un entero distinto de cero. Ver la función get_ident(). Los identificadores de hilos pueden ser reciclados cuando un hilo finaliza y otro hilo es creado. El identificador está disponible incuso después de que el hilo ha abandonado.

native_id

La ID integral nativa de este hilo. Es un entero no negativo, o None si el hilo no ha sido iniciado. Ver la función get_native_id(). Ésta representa la *Thread ID* (TID) tal como haya sido asignada al hilo por el SO (*kernel*). Su valor puede puede ser utilizado para identificar específicamente a este hilo en particular a través de todo el sistema (hasta que el hilo termina, luego de lo cual el valor puede ser reciclado por el SO).

Nota

Similar a las *Process IDs*, las *Thread IDs* sólo son válidas (garantizadas como únicas a través de todo el sistema) desde el momento en que se crea el hilo hasta que el hilo es finalizado.

Disponibilidad: Requiere la función get_native_id().

Nuevo en la versión 3.8.

is_alive()

Retornar si acaso el hilo está vivo.

Este método retorna True desde justo antes de que el método run() inicie hasta junto antes de que el método run() termine. La función enumerate() del módulo retorna una lista de todos los hilos vivos.

daemon

Un valor booleano que indica si este hilo es un hilo demonio (*True*) o no (*False*). Debe ser establecido antes de que se llame a start(), de lo contrario se lanzará un RuntimeError. Su valor inicial se hereda del hilo creador; el hilo principal no es un hilo demonio y por lo tanto todos los hilos creados en el hilo principal tienen por defecto un valor daemon = False.

El programa de Python en su totalidad finaliza cuando no queda ningún hilo no-demonio vivo.

isDaemon()

setDaemon()

Antigua API *getter/setter* para daemon; úsese en cambio directamente como una propiedad.

Objetos tipo *lock*

Una primitiva *lock*, es una primitiva de sincronización que no pertenece a ningún hilo en particular cuando está cerrado. En Python, es la primitiva de sincronización de más bajo nivel actualmente disponible, implementado directamente por el módulo de extensión _thread.

Una primitiva *lock* está en uno de dos estados, «cerrado» o «abierto» (*locked*/*unlocked*). Se crea en estado abierto. Tiene dos métodos básicos, acquire() (adquirir) y release() (liberar). Cuando el estado es *abierto*, acquire() cambia el estado a cerrado y retorna inmediatamente. Cuando el estado es *cerrado*, acquire() bloquea hasta que una llamada a release() en otro hilo lo cambie a abierto, luego la llamada a acquire() lo restablece a cerrado y retorna. El método release() sólo debe ser llamado en el estado cerrado; cambia el estado a abierto y retorna inmediatamente. Si se realiza un intento de liberar un *lock* abierto, se lanzará un RuntimeError.

Los *locks* también soportan el protocolo de gestión de contexto.

Cuando más de un hilo está bloqueado en acquire() esperando que el estado sea abierto, sólo un hilo procederá cuando una llamada a release() restablezca el estado a abierto; cuál de los hilos en espera procederá no está definido, y puede variar a través de las implementaciones.

Todos los métodos se ejecutan de manera atómica.

class threading.**Lock**

La clase que implemente los objetos de la primitiva *lock*. Una vez que un hilo ha adquirido un *lock*, intentos subsecuentes por adquirirlo bloquearán, hasta que sea liberado; cualquier hilo puede liberarlo.

Nótese que Lock es una función de fábrica que retorna una instancia de la versión más eficiente de la clase *Lock* concreta soportada por la plataforma.

acquire(*blocking=True*, *timeout=-1*)

Adquirir un *lock*, bloqueante o no bloqueante.

Cuando se invoca con el argumento *blocking* establecido como True (el valor por defecto), bloquea hasta que el *lock* se abra, luego lo establece como cerrado y retorna True.

Cuando es invocado con el argumento *blocking* como False, no bloquea. Si una llamada con *blocking* establecido como True bloqueara, retorna Falso inmediatamente; de otro modo, cierra el *lock* y retorna True.

Cuando se invoca con el argumento de punto flotante *timeout* fijado a un valor positivo, bloquea por a lo más el número de segundos especificado en *timeout* y mientras el *lock* no pueda ser adquirido. Un argumento *timeout* de «-1» especifica una espera ilimitada. No está admitido especificar un *timeout* cuando *blocking* es falso.

El valor de retorno es True si el *lock* es adquirido con éxito, Falso si no (por ejemplo si *timeout* expiró).

Distinto en la versión 3.2: El parámetro *timeout* es nuevo.

Distinto en la versión 3.2: La adquisición de un *lock* ahora puede ser interrumpida por señales en POSIX si la implementación de hilado subyacente lo soporta.

release()

Libera un *lock*. Puede ser llamado desde cualquier hilo, no solo el hilo que ha adquirido el *lock*.

Cuando el *lock* está cerrado, lo restablece a abierto, y retorna. Si cualquier otro hilo está bloqueado esperando que el *lock* se abra, permite que exactamente uno de ellos proceda.

Cuando se invoca en un *lock* abierto, se lanza un RuntimeError.

No hay valor de retorno.

locked()

Retorna *true* si el *lock* ha sido adquirido.

Objetos *Rlock*

Un *lock* reentrante es una primitiva de sincronización que puede ser adquirido múltiples veces por el mismo hilo. Internamente, utiliza el concepto de «hilo dueño» y «nivel de recursividad»

además del estado abierto/cerrado utilizado por las primitivas *locks*. Si está en estado cerrado, algún hilo es dueño del *lock*; si está en estado abierto, ningún hilo es dueño.

Para cerrar el *lock*, un hilo llama a su método acquire(); esto retorna una vez que el hilo se ha adueñado del *lock*. Para abrir el *lock*, un hilo llama a su método release(). Pares de llamadas acquire()/release() pueden anidarse; sólo el release() final (el release() del par más externo) restablece el *lock* a abierto y permite que otro hilo bloqueado en acquire() proceda.

Los *locks* reentrantes también soportan el protocolo de manejo de contextos.

class threading.**RLock**

Esta clase implementa objetos tipo *lock* reentrantes. Un *lock* reentrante debe ser liberado por el hilo que lo adquirió. Una vez que un hilo ha adquirido un *lock* reentrante, el mismo hilo puede adquirirlo otra vez sin bloquearse; el hilo debe liberarlo una vez por vez que lo adquiere.

Nótese que RLock en realidad es una función fábrica que retorna una instancia de la versión más eficiente de la clase RLock concreta que sea soportada por la plataforma.

acquire(*blocking=True*, *timeout=-1*)

Adquirir un *lock*, bloqueante o no bloqueante.

Cuando se invoca sin argumentos: si este hilo ya es dueño del *lock*, incrementa el nivel de recursividad en uno, y retorna inmediatamente. De otro modo, si otro hilo es dueño del *lock*, bloquea hasta que se abra el *lock*. Una vez que el *lock* se abra (ningún hilo sea su dueño), se adueña, establece el nivel de recursividad en uno, y retorna. Si más de un hilo está bloqueado esperando que sea abra el *lock*, solo uno a la vez podrá apoderarse del *lock*. No hay valor de retorno en este caso.

Cuando se invoca con el argumento *blocking* fijado en *true*, hace lo mismo que cuando se llama sin argumentos y retorna True.

Cuando se invoca con el argumento *blocking* fijado a falso, no bloquea. Si una llamada sin argumento bloquease, retorna False inmediatamente; de otro modo, hace lo mismo que al llamarse sin argumentos, y retorna True.

Cuando se invoca con el argumento de coma flotante *timeout* fijado a un valor positivo, bloquea por máximo el número de segundos especificado por *timeout* y mientras el *lock* no pueda ser adquirido. Retorna True si el *lock* ha sido adquirido, falso si el tiempo de espera *timeout* ha caducado.

Distinto en la versión 3.2: El parámetro *timeout* es nuevo.

release()

Libera un *lock*, disminuyendo el nivel de recursividad. Si después de la disminución es cero, restablece el *lock* a abierto (no perteneciente a ningún hilo), y si cualquier otro hilo está bloqueado esperando que se abra el *lock*, permite que exactamente uno de ellos proceda. Si luego de la disminución el nivel de recursividad todavía no es cero, el *lock* permanece cerrado y perteneciente al hilo llamador.

Solo llámese este método cuando el hilo llamador sea dueño del *lock*. Se lanza un RuntimeError si se llama este método cuando el *lock* esta abierto.

No hay valor de retorno.

Objetos condicionales

Una condición variable siempre va asociada a algún tipo de *lock*. éste puede ser provisto o se creará uno por defecto. Proveer uno es útil cuando varias variables de condición deben compartir el mismo *lock*. El *lock* es parte del objeto condicional: no es necesario rastrearlo por separado.

Una condición variable obedece el protocolo de gestión de contexto: al usar la declaración with se adquiere el *lock* asociado por la duración del bloque contenido. Los métodos acquire() y release()también llaman los métodos correspondientes del *lock* asociado.

Otros métodos deben llamarse con el *lock* asociado conservado. El método wait() libera el *lock*, y luego bloquea hasta que otro hilo lo despierte llamando notify() o notify_all(). Una vez que ha sido despertado, wait() re-adquiere el *lock* y retorna. También es posible especificar un tiempo de espera.

El método notify() despierta a uno de los hilos que esperan a la condición variable, si es que alguno espera. El método notify_all() despierta a todos los hilos que estén esperando a la condición variable.

Nota: Los métodos notify() y notify_all() no liberan el *lock*; esto significa que el hilo o los hilos que han sido despertados no retornaran de su llamada de wait() inmediatamente, sino solo una vez que el hilo que haya llamado a notify() o notify_all() renuncie finalmente a la propiedad del *lock*.

El estilo típico de programación con variables condicionales utiliza el *lock* para sincronizar el acceso a algún estado compartido; hilos que estén interesados en un cambio de estado en particular llamarán a wait() reiteradamente hasta que vean el estado deseado, mientras que los hilos que modifiquen el estado llamarán a notify() o a notify_all() cuando cambien el estado de modo que pudiera ser que el el estado sea el deseado por alguno de los hilos en espera. Por ejemplo, el siguiente código es una situación genérica de productor-consumidor con capacidad de búfer ilimitada:

```
# Consume one item
with cv:
    while not an_item_is_available():
        cv.wait()
    get_an_available_item()

# Produce one item
with cv:
    make_an_item_available()
    cv.notify()
```

El bucle while que verifica la condición de la aplicación es necesario porque wait() puede retornar después de una cantidad arbitraria de tiempo, y la condición que dio pie a la llamada de notify()puede ya no ser verdadera. Esto es inherente a la programación multi-hilo. El método wait_for()puede usarse para automatizar la revisión de condiciones, y facilita la computación de tiempos de espera:

```
# Consume an item
with cv:
    cv.wait_for(an_item_is_available)
    get_an_available_item()
```

Para elegir entre notify() y notify_all(), considérese si un cambio de estado puede ser interesante para uno o varios hilos en espera. Por ejemplo en una típica situación productor-consumidor, agregar un elemento al búfer sólo necesita despertar un hilo consumidor.

class threading.**Condition**(*lock=None*)

Esta clase implementa objetos de condición variable. Una condición variable permite que uno o más hilos esperen hasta que sean notificados por otro hilo.

Si se provee un argumento *lock* distinto de None, debe ser un objeto Lock o RLock, y se utiliza como el *lock* subyacente. De otro modo, se crea un nuevo objeto RLock y se utiliza como el *lock*subyacente.

Distinto en la versión 3.3: cambiado de función de fábrica a una clase.

acquire(**args*)

Adquiere el *lock* subyacente. Este método llama al método correspondiente sobre el *lock* subyacente; el valor de retorno es lo que retorne aquel método.

release()

Libera el *lock* subyacente. Este método llama al método correspondiente en el *lock* subyacente; no tiene valor de retorno.

wait(*timeout=None*)

Espera hasta ser notificado o hasta que el tiempo de espera caduque. Si el hilo invocador no ha adquirido el *lock* cuando este método es llamado, se lanza un RuntimeError.

Este método libera el *lock* subyacente, y luego bloquea hasta ser despertado por una llamada a notify() o notify_all() para la misma condición variable en otro hilo, o hasta que el tiempo de espera opcional se cumpla. Una vez que ha sido despertado o el tiempo de espera ha pasado, re-adquiere el *lock* y retorna.

Cuando haya un argumento *timeout* presente y no sea None, debe ser un número de punto flotante que especifique un tiempo de espera para la operación en segundos (o fracciones de segundo).

Cuando el *lock* subyacente es un RLock, no se libera utilizando su método release(), ya que esto podría no abrir realmente el *lock* cuando haya sido adquirido múltiples veces recursivamente. En cambio, se usa una interfaz interna de la clase RLock, que lo abre realmente incluso cuando haya sido adquirido múltiples veces recursivamente. Otra interfaz interna se usa luego para restablecer el nivel de recursividad cuando el *lock* es readquirido.

El valor de retorno es True a menos que un *timeout* dado haya expirado, en cuyo caso será False.

Distinto en la versión 3.2: Previamente, el método siempre retornaba None.

wait_for(*predicate*, *timeout=None*)

Espera a que una condición se evalúe como verdadera. *predicate* debe ser un invocable cuyo resultado se interpretará como un valor booleano. Se puede proveer un *timeout* que especifique el máximo tiempo de espera.

Este método utilitario puede llamar a wait() reiteradas veces hasta que se satisfaga el predicado, o hasta que la espera caduque. El valor de retorno es el último valor de retorno del predicado y se evaluará a False si el método ha caducado.

Al ignorar la propiedad *feature*, llamar a este método equivale vagamente a escribir:

```
while not predicate():
    cv.wait()
```

Por ende, aplican las mismas reglas que con wait(): El *lock* debe ser conservado cuando se llame y es re-adquirido al momento del retorno. El predicado se evalúa con el *lock* conservado.

Nuevo en la versión 3.2.

notify(*n=1*)

Por defecto, despierta a un hilo que esté esperando por esta condición, si lo existe. Si el hilo llamador no ha adquirido el *lock* cuando se llama este método, se lanza un RuntimeError.

Este método despierta como máximo *n* de los hilos que estén esperando por la condición variable; no es una opción si no hay hilos esperando.

La implementación actual despierta exactamente *n* hilos, si hay por lo menos *n* hilos esperando. Sin embargo, no es seguro apoyarse en este comportamiento. A futuro, una implementación optimizada podría ocasionalmente despertar a más de *n* hilos.

Nota: un hilo que ha sido despertado no retorna realmente de su llamada a wait() hasta que pueda readquirir el *lock*. Ya que notify() no libera el *lock*, su llamador debiera hacerlo.

notify_all()

Despierta a todos los hilos que esperen por esta condición. Este método actúa como notify(), pero despierta a todos los hilos en espera en vez de a uno. Si el hilo llamador no ha adquirido el *lock* cuando se llama a este método, se lanza un RuntimeError.

Objetos semáforo

Éste es uno de las primitivas de sincronización más antiguos en la historia de las ciencias de la computación, inventado por el pionero en ciencias de la computación holandés Edsger W. Dijkstra (él utilizó los nombres P() y V() en lugar de acquire() y release())

Un semáforo administra un contador interno que se disminuye por cada llamada a acquire() y se incrementa por cada llamada a release(). El contador no puede bajar de cero; cuando acquire() lo encuentra en cero, bloquea, esperando hasta que otro hilo llame release().

Los semáforos también tienen soporte para el protocolo de gestión de contexto.

class threading.**Semaphore**(*value=1*)

Esta clase implementa los objetos semáforo. Un semáforo gestiona un contador atómico que representa el número de llamadas a release() menos el número de llamadas a acquire(), más un valor inicial. El método acquire() bloquea si es necesario, hasta que pueda retornar sin volver el contador negativo. Si no es provisto, el valor por defecto de *value* será 1.

El argumento opcional da el *value* inicial al contador interno; por defecto es 1. Si el *value* provisto es menor a 0; se lanza un ValueError.

Distinto en la versión 3.3: cambiado de función de fábrica a una clase.

acquire(*blocking=True*, *timeout=None*)

Adquirir un semáforo.

Cuando se invoca sin argumentos:

- Si el contador interno es mayor a cero de entrada, lo disminuye en uno y retorna Trueinmediatamente.
- Si el contador interno es cero de entrada, bloquea hasta ser despertado por una llamada a release(). Una vez despierto (y el contador sea mayor a 0), disminuye el contador en 1 y retorna True. Se despertará exactamente un hilo por cada llamada a release(). No debiese confiarse en el orden en que los hilos sean despertados.

Cuando se invoca con *blocking* fijado en falso, no bloquea. Si una llamada sin un argumento bloquease, retorna Falso inmediatamente; de otro modo, hace lo mismo que cuando se llama sin argumentos, y retorna True.

Cuando se invoca con *timeout* distinto de None, bloqueará por un tiempo máximo en segundos fijados en *timeout*. Si *acquire* no se completa exitosamente en ese intervalo, retorna False. De otro modo retorna True.

Distinto en la versión 3.2: El parámetro *timeout* es nuevo.

release()

Libera un semáforo, incrementando el contador interno en uno. Cuando éste haya sido cero de entrada y otro hilo esté esperando que sea nuevamente mayor a cero, despierta a ese hilo.

class threading.**BoundedSemaphore**(*value=1*)

Clase que implementa objetos de semáforo delimitados. Un semáforo delimitado verifica que su valor actual no exceda su valor inicial. Si lo hace, se lanza un ValueError. En la mayoría de las situaciones se utilizan los semáforos para cuidar recursos con capacidad limitada. Si se libera el semáforo demasiadas veces es signo de un *bug*. Si no se provee, el valor por defecto de *value* será 1.

Distinto en la versión 3.3: cambiado de función de fábrica a una clase.

Ejemplo de Semaphore

Los semáforos suelen utilizarse para cuidar recursos con capacidad limitada, por ejemplo, un servidor de base de datos. En cualquier situación en que el tamaño de los recursos sea fijo, se debe usar un semáforo delimitado. Antes de generar cualquier hilo de trabajo, tu hilo principal debe inicializar el semáforo:

```
maxconnections = 5
# ...
pool_sema = BoundedSemaphore(value=maxconnections)
Una vez que han sido generados, los hilos de trabajo llaman a
los métodos acquire y release cuando necesitan conectarse al
servidor:
with pool_sema:
    conn = connectdb()
    try:
        # ... use connection ...
    finally:
        conn.close()
```

El uso de semáforos delimitados reduce la posibilidad de que pase inadvertido un error de programación que cause que el semáforo sea liberado más veces de las que sea adquirido.

Objetos de eventos

Éste es uno de los mecanismos más simples de comunicación entre hilos: un hilo señala un evento y otro hilo lo espera.

Un objeto de evento maneja una marca interna que puede ser establecida como verdadera mediante el método set() y restablecida a falsa mediante el método clear(). El método wait() bloquea hasta que la marca sea *true*.

class threading.**Event**

Clase que implementa los objetos de evento. Un evento gestiona un indicador que puede ser establecido a verdadero mediante el método set() y restablecido a falso con el método clear(). El método wait() bloquea hasta que el indicador sea verdadero. El indicador es inicialmente falso.

Distinto en la versión 3.3: cambiado de función de fábrica a una clase.

is_set()

Retorna True exclusivamente si el indicador interno es verdadero.

set()

Establece el indicador interno a verdadero. Todos los hilos que estén esperando que se vuelva verdadero serán despertados. Los hilos que llaman a wait() una vez que el indicador marca verdadero no bloquearán.

clear()

Restablece el indicador a falso. Posteriormente, los hilos que llamen a wait() bloquearán hasta que se llame a set() para establecer el indicador interno a verdadero nuevamente.

wait(*timeout=None*)

Bloquea hasta que el indicador interno sea verdadero. Si el indicador interno es verdadero de entrada, retorna inmediatamente. De otro modo, bloquea hasta que otro hilo llame a set()para establecer el indicador a verdadero, o hasta que el tiempo de espera opcional caduque.

Cuando se presenta un argumento para el tiempo de espera *timeout* distinto de None, debe ser un número de punto flotante que especifique un tiempo de espera para la operación en segundos (o fracciones en su defecto).

Este método retorna True exclusivamente si el indicador interno ha sido establecido a verdadero, ya sea antes de la llamada a la espera o después de que la espera inicie, por lo que siempre retorna True excepto si se provee un tiempo de espera máximo y la operación caduca.

Distinto en la versión 3.1: Previamente, el método siempre retornaba None.

Objetos temporizadores

Esta clase representa una acción que sólo debe ejecutarse luego de que una cierta cantidad de tiempo transcurra — un temporizador. Timer es una subclase de Thread y en tanto tal también funciona como un ejemplo de creación de hilos personalizados.

Los temporizadores son iniciados, tal como los hilos, al llamarse su método start(). El temporizador puede ser detenido (antes de que su acción haya comenzado) al llamar al método cancel(). El intervalo que el temporizador esperará antes de ejecutar su acción puede no ser exactamente el mismo que el intervalo especificado por el usuario.

Por ejemplo:

```
def hello():
    print("hello, world")

t = Timer(30.0, hello)
t.start()  # after 30 seconds, "hello, world" will be printed
class threading.Timer(interval, function, args=None, kwargs=None
)
```

Crear un temporizador que ejecutará *function* con los argumentos *args* y los argumentos de palabra clave *kwargs*, luego de que una cantidad *interval* de segundos hayan transcurrido. Si *args* es None (por defecto) se utilizará una lista vacía. Si *kwargs* es None (por defecto) se utilizará un *dict*vacío.

Distinto en la versión 3.3: cambiado de función de fábrica a una clase.

cancel()

Detiene el temporizador, y cancela la ejecución de la acción del temporizador. Esto sólo funcionará si el temporizador está en etapa de espera.

Objetos de barrera

Nuevo en la versión 3.2.

Esta clase provee una primitiva de sincronización simple para ser usado por un número fijo de hilos que necesitan esperarse entre ellos. Cada uno de los hilos intenta pasar la barrera llamando al método wait() y bloqueará hasta que todos los hilos hayan hecho sus respectivas llamadas a wait(). En este punto, los hilos son liberados simultáneamente.

La barrera puede ser reutilizada cualquier número de veces para el mismo número de hilos.

Como ejemplo, aquí hay una manera simple de sincronizar un hilo cliente con uno servidor:

```
b = Barrier(2, timeout=5)

def server():
    start_server()
    b.wait()
    while True:
        connection = accept_connection()
        process_server_connection(connection)

def client():
    b.wait()
    while True:
        connection = make_connection()
        process_client_connection(connection)
```

class threading.**Barrier**(*parties*, *action=None*, *timeout=None*)

Crear un objeto de barrera para un número *parties* de hilos. Una *action*, si es provista, es un invocable a ser llamado por uno de los hilos cuando sean liberados. *timeout* es el valor de tiempo de espera máximo por defecto si no se especifica uno en el método wait().

wait(*timeout=None*)

Pasa la barrera. Cuando todos los hilos involucrados en el objeto barrera han llamado esta función, se liberan todos simultáneamente. Si se provee un valor *timeout*, se utilizará con preferencia sobre cualquiera que haya sido suministrado al constructor de la clase.

El valor de retorno es un entero en el rango desde 0 hasta *parties* – 1, diferente para cada hilo. Puede ser utilizado para seleccionar a un hilo para que haga alguna limpieza especial, por ejemplo:

```
i = barrier.wait()
if i == 0:
    # Only one thread needs to print this
    print("passed the barrier")
```

Se se provee una *action* al constructor, uno de los hilos la habrá llamado antes de ser liberado. Si acaso esta llamada lanzara un error, la barrera entra en estado *broken* (roto).

Si la llamada caduca, la barrera entra en estado *broken*.

Este método podría lanzar una excepción BrokenBarrierError si la barrera está rota o si se reinicia mientras el hilo está esperando.

reset()

Retorna la barrera al estado por defecto, vacío. Cualquier hilo que esté a su espera recibirá la excepción BrokenBarrierError.

Nótese que utilizar esta función podría requerir alguna sincronización externa si existen otros hilos cuyos estados sean desconocidos. Si una barrera se rompe puede ser mejor abandonarla y crear una nueva.

abort()

Coloca la barrera en estado roto. Esto causa que cualquier llamada activa o futura a wait()falle con el error BrokenBarrierError. Úsese por ejemplo si uno de los hilos necesita abortar, para evitar que la aplicación quede en punto muerto.

Puede ser preferible simplemente crear la barrera con un valor *timeout* sensato para cuidarse automáticamente de que uno de los hilos falle.

parties

El número de hilos requeridos para pasar la barrera.

n_waiting

El número de hilos actualmente esperando en la barrera.

broken

Un valor booleano que será True si la barrera está en el estado roto.

exception threading.**BrokenBarrierError**

Esta excepción, una subclase de RuntimeError, se lanza cuando el objeto Barrier se restablece o se rompe.

Uso de *locks*, condiciones y semáforos en la declaración with

Todos los objetos provistos por este módulo que tienen métodos acquire() y release() pueden ser utilizados como administradores de contexto para una declaración with. El método acquire() será llamado cuando se ingresa al bloque y el método release() será llamado cuando se abandona el bloque. De ahí que, el siguiente fragmento:

```
with some_lock:
    # do something...
```

sea equivalente a:

```
some_lock.acquire()
try:
    # do something...
finally:
    some_lock.release()
```

Actualmente, los objetos Lock, RLock, Condition, Semaphore, y BoundedSemaphore pueden ser utilizados como gestores de contexto con declaraciones with.